

Проект 1. Знакомство с Python

В этом проекте ты познакомишься с основами Python, изучишь синтаксис языка, научишься работать с переменными, типами данных, условиями и циклами.

Содержание

- Глава 1
 - Введение в Python и основы синтаксиса
 - Встроенные функции
 - Типы данных
 - Математические операции
 - Циклы
 - Словарь
- Глава 2
 - Практические задания
 - Задание 1. Считаем, сколько пользователей в списке данных
 - Задание 2. Выводим информацию о каждом пользователе
 - Задание 3. Создаем список, который будет содержать только конкретных пользователей

Глава 1

Введение в Python и основы синтаксиса

BI-инструменты, такие как DataLens, хорошо подходят для визуализации метрик, построения дашбордов. Но в работе аналитика возникают задачи, для решения которых приходится подключать Python.

Python — это один из самых популярных языков, который используют аналитики данных. Он удобен тем, что на нем легко начать работать, код читается как обычный текст на английском, а многие задачи решаются в несколько строк. С помощью Python аналитик может быстро проверять идеи, считать метрики, обрабатывать данные и строить графики.

Перед тем как начать писать код, нужно установить и настроить IDE (Integrated Development Environment/Интегрированная среда разработки).

IDE — это редактор, который не только позволяет писать код, но и автоматически исправляет ошибки, подсказывает и сразу показывает, как будет выглядеть результат.

IDE бывают разные, но в этом курсе ты будешь работать с Visual Studio Code (VS Code)

VS Code — это бесплатный, удобный и современный редактор. VS Code помогает быстрее находить и исправлять ошибки, а дополнительные расширения, в том числе AI-помощник, делают работу еще проще. Еще одно важное преимущество VS Code — универсальность, в нем можно работать не только с Python, но и с другими языками программирования.

Чтобы приступить к работе, установи Python и VS Code по инструкции, которую ты найдешь в прикрепленных файлах на Платформе. В папке src скачай ноутбук project_5.ipynb, это твоя рабочая тетрадь на проект, для удобства в ней можно работать и запускать код.

Когда все установлено и работает, можно переходить к основам. Вот несколько правил, которые в дальнейшем помогут тебе писать код правильно.

1. Создание переменной и присвоение ей значений. Переменная — это контейнер для хранения данных, она может хранить различные типы данных, такие как числа, строки, списки и так далее. Имя переменной должно начинаться с буквы или символа подчеркивания «_». Название переменной должно быть очевидно. Например, если ты анализируешь активность пользователей, назови переменную `user_activity`, а если анализируешь общую сумму чека, то назови `total_bill`.
2. Комментарии. Они всегда начинаются с символа «#». Комментарии помогают другим разработчикам и тебе самому понять, что делает код. Это особенно важно в больших проектах, где код может быть сложным и трудночитаемым.
3. Кавычки. Можно использовать как одинарные ('), так и двойные (") кавычки для создания строк. Используй одинарные, если строка содержит двойные кавычки (например, 'He said "Hello!"'), и, наоборот, используй двойные, если внутри строки есть одинарные кавычки (например, "It's a nice day"). Главное правило — начинать и заканчивать строку одним и тем же типом кавычек.

Ниже ты можешь увидеть примеры кода и результат, который получается по итогу.

```
# Зададим переменную и ее значение:  
# x — переменная;  
# [1, 2, 3] — значение переменной.  
  
# Вывод:  
x = [1, 2, 3]
```

```
# Зададим переменную для хранения списка задач:  
# tasks — переменная;  
# ["Сделать отчет", "Провести встречу", "Ответить на письма"] — значение  
переменной.  
  
# Вывод:  
tasks = ["Сделать отчет", "Провести встречу", "Ответить на письма"]
```

Встроенные функции

В Python уже есть заранее готовые действия, которые можно использовать, как на калькуляторе. Это как готовые инструменты, которые не нужно создавать самому — они уже есть. Эти инструменты называются встроенными функциями. Аналитик данных работает с большим количеством информации, и встроенные функции помогают ему быстро обрабатывать эту информацию.

Функция	Что делает	Пример данных	Пример в аналитике
len(x)	Считает сколько элементов	len([10, 12, 8]) → 3	Подсчет количества пользователей в списке
sum(x)	Считает сумму чисел	sum([10, 12, 8]) → 30	Суммирование времени, проведенного пользователями в приложении
min(x)	Находит минимум	min([10, 12, 8]) → 8	Поиск минимального времени, проведенного пользователем в приложении
max(x)	Находит максимум	max([10, 12, 8]) → 12	Поиск максимального времени, проведенного пользователем в приложении
sorted(x)	Сортирует список	sorted([3,1,2]) → [1,2,3]	Сортировка пользователей по времени, проведенному в приложении
round(x, n)	Округляет значения до указанного количества знаков после запятой	round(3.14159, 2) → 3.14	Округление среднего времени, проведенного пользователем в приложении
type(x)	Выводит тип данных	type(10) → int	Проверка типа данных, чтобы убедиться, что они корректны для анализа
print(x)	Показывает результат на экране	print("Привет") → Привет	Вывод результатов анализа на экран для проверки

```
# Подсчитаем количество пользователей приложения в списке:
users = [10, 12, 8, 15]
result = len(users)
print(result)

# Вывод:
4
```

```
# Округлим время, проведенное пользователем в приложении:
time = 3.14159
result = round(time, 2)
print(result)

# Вывод:
3.14
```

Типы данных

Когда аналитик работает с Python, ему важно понимать, какие бывают типы данных, потому что компьютер по-разному работает с разными значениями. Каждый тип данных нужен для разных задач. Например, если ты хочешь посчитать сумму денег, тебе нужны числа, а если ты хочешь записать имя человека, тебе нужна строка. Изучи таблицу с основными типами данных: в ней ты можешь увидеть английское название типов данных, как обозначается конкретный тип в Python, а также примеры данных и где конкретный тип чаще всего используется.

Тип данных	Английское название	Обозначение в Python	Примеры	Где встречается в аналитике
Целые числа	integer	int	5, 10, -3	количество заказов, кликов, пользователей
Числа с дробью	floating point	float	3.14, 0.5, -1.2	деньги, проценты, средние значения
Строки	string	str	"Иван", "Москва", "2024-01-10"	город, категория, дата как текст
Логические значения	boolean	bool	True, False	оплачено / не оплачено, активен / не активен
Пропуск	null	None	None	пропущенные значения в данных
Список	list	list	[1, 2, 3], ["яблоко", "груша"]	много значений в одной ячейке

```
# Зададим переменную для хранения имени пользователя:
# user_name — переменная;
# "Иван" — значение переменной (имя пользователя).
user_name = "Иван"
print(user_name)

# Вывод:
Иван
```

```
# Выведем тип переменной user_name:
type(user_name)

# Вывод:
str
```

Математические операции

Математические операции в Python — это одна из основ языка программирования, которая позволяет выполнять различные арифметические действия. Если нужно быстро подсчитать общую стоимость товаров в магазине, определить скидку или рассчитать средний балл студентов — все это делается простыми арифметическими операциями.

Приоритет	Оператор Python	Операция	Пример
1	**	Возведение в степень	5 ** 5
2	%	Деление по модулю (получение остатка)	16 % 7
3	//	Целочисленное деление (дробная часть отбрасывается)	13 // 3
4	/	Деление	39 / 2
5	*	Умножение	123 * 321
6	-	Вычитание	999 - 135
7	+	Сложение	478 + 32

```
# Рассчитаем общую стоимость товаров, если известна стоимость товара (a) и
# количество проданных товаров (b):
a = 200
b = 3
print(a * b)

# Вывод:
500
```

```
# Рассчитаем объем коробки, чтобы понять, сколько товаров поместится в одну
# коробку. Возведем число 56 в степень 3:
x = 56**3
print(x)

# Вывод:
175616
```

Циклы

Циклы в программировании — это конструкции, которые позволяют многократно выполнять один и тот же блок кода. Это особенно полезно, когда нужно обработать большое количество данных или выполнить одно и то же действие несколько раз.

Представь, что у тебя есть длинный список из 10 000 строк, ты явно не сможешь обработать его руками. В этом случае цикл поможет пройти по всем значениям и выполнить одно и то же действие.

В Python есть два основных типа циклов: `for` и `while`. `While` используется для выполнения блока кода до тех пор, пока условие истинно. Этот тип нужен для работы с более сложными данными, в данном курсе мы его использовать не будем, поэтому далее рассмотрим только цикл `for`.

Представь, что ты работаешь с приложением магазина по доставке еды. Тебе необходимо отобразить список товаров на странице, для этого нужно проверить наличие каждого товара. Берешь первый товар — проверяешь, затем второй, третий и так далее, пока не дойдешь до конца списка. Чтобы не делать это вручную, воспользуемся циклом `for`, который пройдет по всему списку и все проверит.

```
# Проверяем список товаров:
x = ["молоко", "хлеб", "яйца", "бананы", "вода"]
for product in x:
    print(product)

# Вывод:
молоко
хлеб
яйца
бананы
вода
```

Словарь

Словари (dictionary) в Python — это коллекции, которые хранят данные в виде пар «ключ–значение». Они позволяют быстро находить значения по ключу, что делает их очень удобными для работы с данными.

Что хотим сделать	Как в Python
Создать словарь	<code>d = {}</code>
Получить значение из словаря по конкретному ключу. Например, получить цену плана premium.	<code>d[key]</code>
Получить значение из словаря по ключу, но если ключа нет, то вернуть какое-то значение по умолчанию. Например, если ключа "C" нет, то вернуть 0.	<code>d.get(key, default)</code>
Получить список всех ключей или значений из словаря. Например, получить список всех планов подписки.	<code>d.keys()</code> , <code>d.values()</code>
Пройти по каждой паре «ключ–значение» в словаре и выполнить какое-то действие. Например, вывести на экран все планы подписки и их цены.	<code>for k, v in d.items():</code>

```
# Разберем пример и создадим словарь активности пользователей. Каждый
# пользователь имеет следующие атрибуты:
# - name — это имя пользователя;
# - downloaded — это флаг, указывающий, скачал ли пользователь приложение;
# - purchased — это флаг, указывающий, совершил ли пользователь покупку в
```

```
приложении.  
user_activity = {  
    "user1": {  
        "name": "Alice",  
        "downloaded": True,  
        "purchased": True  
    },  
    "user2": {  
        "name": "Gena",  
        "downloaded": True,  
        "purchased": False  
    },  
    "user3": {  
        "name": "Oleg",  
        "downloaded": True,  
        "purchased": False  
    }  
}
```

```
# Получим имя пользователя по конкретному ключу "user1":  
user_data = user_activity["user1"]  
print(user_data["name"])  
  
# Вывод:  
Alice
```

Глава 2

Практические задания

Тебе предстоит поработать с реальными данными. Ниже ты найдешь список активности пользователей в приложении по изучению английского языка. Тебе нужно выполнить несколько заданий и проанализировать результаты. Каждый пользователь имеет следующие атрибуты:

- **name** — имя пользователя.
- **downloaded** — флаг, указывающий, скачал ли пользователь приложение.
- **purchased** — флаг, указывающий, совершил ли пользователь покупку в приложении.
- **time_spent** — время, проведенное пользователем в приложении (в минутах).
- **last_activity** — дата последней активности.
- **viewed** — просмотрен контент или нет.

```
users = [  
    {"name": "Alice", "downloaded": True, "purchased": True, "time_spent":  
120, "last_activity": "2023-10-01", "viewed": True},  
    {"name": "Gena", "downloaded": True, "purchased": False, "time_spent":  
60, "last_activity": "2023-09-30", "viewed": True},  
    {"name": "Oleg", "downloaded": True, "purchased": False, "time_spent":  
10, "last_activity": "2023-10-02", "viewed": False},  
]
```

```
[{"name": "Anna", "downloaded": False, "purchased": False, "time_spent": 0, "last_activity": "2023-10-03", "viewed": False}, {"name": "Max", "downloaded": True, "purchased": True, "time_spent": 90, "last_activity": "2023-10-04", "viewed": True}, {"name": "Lena", "downloaded": False, "purchased": False, "time_spent": 0, "last_activity": "2023-10-05", "viewed": False}, {"name": "Ivan", "downloaded": True, "purchased": False, "time_spent": 45, "last_activity": "2023-10-06", "viewed": True}, {"name": "Kate", "downloaded": True, "purchased": True, "time_spent": 150, "last_activity": "2023-10-07", "viewed": True}, {"name": "Sergey", "downloaded": True, "purchased": False, "time_spent": 30, "last_activity": "2023-10-08", "viewed": False}, {"name": "Maria", "downloaded": False, "purchased": False, "time_spent": 0, "last_activity": "2023-10-09", "viewed": False}]
```

Задание 1. Считаем, сколько пользователей в списке данных

1. Создай список `users`, содержащий информацию о пользователях.
2. Используй подходящую функцию для подсчета количества элементов в списке `users`.
3. Выведи результат с помощью функции `print()`.
4. Запусти скрипт в VS Code.
5. Сохрани файл в формате `.ipynb` и загрузи его в папку `src`.

Задание 2. Выводим информацию о каждом пользователе

1. Используй список `users` из предыдущего задания или создай список повторно.
2. Используя цикл, выведи для каждого пользователя в списке его имя и время, проведенное в приложении.
3. Выведи результат с помощью функции `print()`.
4. Запусти скрипт в VS Code.
5. Сохрани файл в формате `.ipynb` и загрузи его в папку `src`.

Задание 3. Создаем список, который будет содержать только конкретных пользователей

1. Используй список `users` из Задания 1 или создай список повторно.
2. Используй цикл для списка `users` и создай список пользователей, которые провели в приложении более 30 минут.
3. Запусти скрипт в VS Code.
4. Сохрани файл в формате `.ipynb` и загрузи его в папку `src`.